

Lumora — Technical Specification v1.0

Oct 3, 2026 · @Mayank Singhal

Overview and conventions

This spec is the build contract for Lumora v1 (MVP + parity). Engineers should be able to build every module from it; anything not specified here is decided by the tech lead and recorded as an ADR in `/docs/adr`. Product intent lives in the companion **Lumora — PRD & Scope of Work**.

Scope of v1: auth, creator KYC, profiles, posts, PPV, subscriptions, bundles, messaging, vault, tips/wallet, live streaming, referrals, agencies, discovery, dashboards, payouts, moderation, admin console.

Conventions

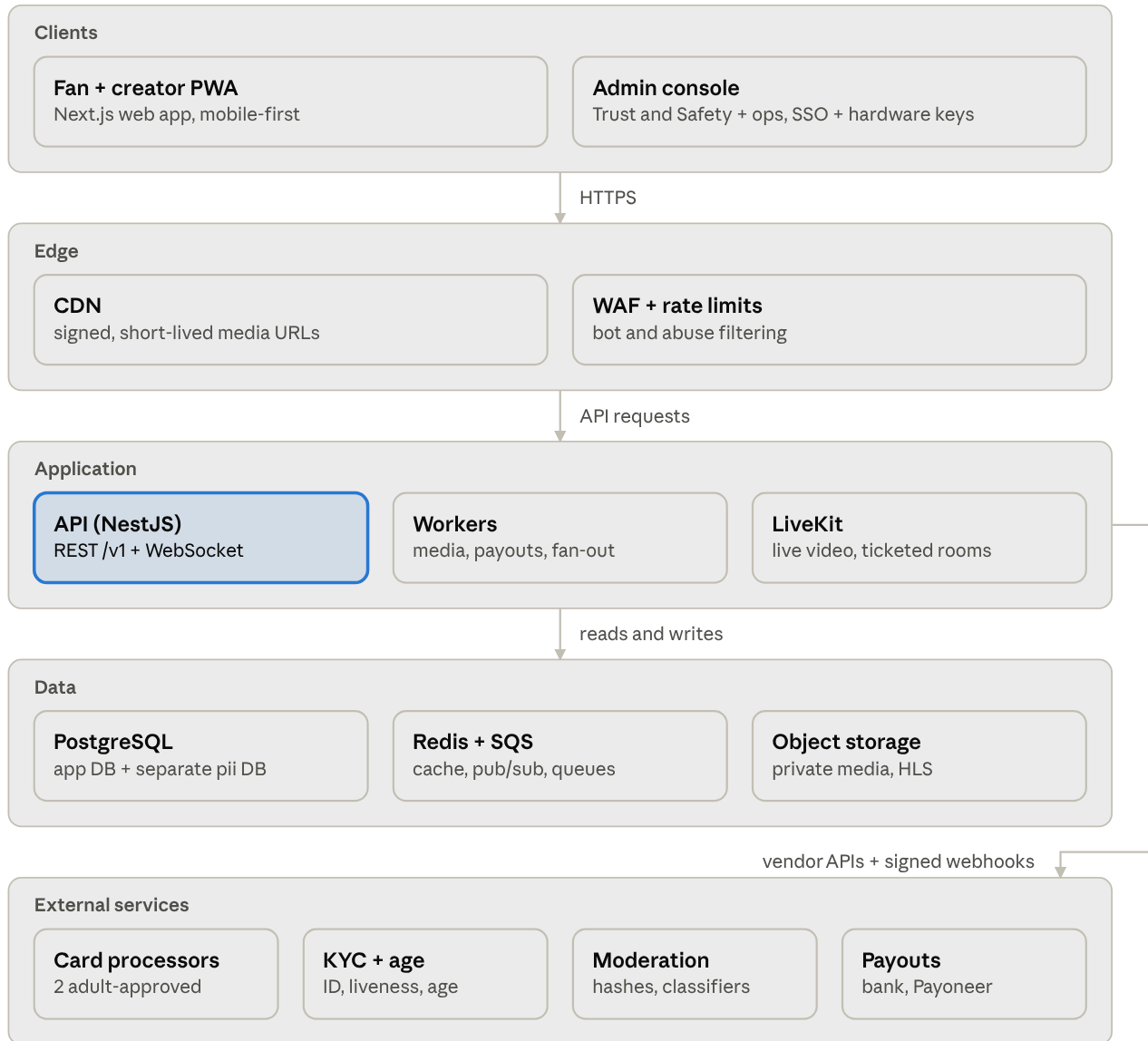
- Language: TypeScript everywhere (strict mode). Node 22 LTS. pnpm workspaces + Turborepo.
- IDs: UUIDv7 (time-sortable) stored as `uuid`. Public handles are separate from IDs.
- Money: integer minor units (`amount_cents bigint`) + ISO-4217 `currency char(3)`. Never floats.
- Time: `timestampz`, UTC in storage and APIs, ISO-8601 strings in JSON.
- API: REST JSON under `/v1`, camelCase fields, cursor pagination, idempotency keys on every money-moving POST.
- Soft delete via `deleted_at`; hard delete only by the data-retention job.
- Every table has `created_at`, `updated_at`. Every privileged action writes to `audit_log`.
- Feature flags (Unleash or GrowthBook) gate every module not yet launched.
- Definition of Done: code reviewed, unit + integration tests passing, OpenAPI updated, migration reversible, metrics + logs added, flag in place, docs updated.

Glossary: PPV = pay-per-view item; Vault = creator media library; GMV = gross fan spend; T&S = Trust & Safety; KYC = identity verification; AV = age verification; Chatter = staff account acting for a creator.

System architecture

A modular monolith: one API service plus background workers, backed by Postgres, Redis and object storage, with all regulated work (cards, identity, CSAM screening, payouts) delegated to specialist vendors.

One API fronts all data; vendors handle payments, KYC, moderation



Lumora system architecture · 5 layers

Clients never reach data stores or vendors directly. Vendors call back only through signed webhooks into the API, which queues work for the workers.

Repository layout, environments, tooling

One monorepo; apps deploy independently, packages are shared.

```
lumora/  
  apps/  
    web/           Next.js 15 (App Router) – fan + creator PWA  
    admin/         Next.js – Trust & Safety + ops console (VPN/SSO only)  
    api/           NestJS – REST /v1 + WebSocket gateway  
    worker/        NestJS standalone – BullMQ/SQS consumers (media, payouts,  
mass messages, moderation)  
    media-hooks/   Lambda handlers – transcoding callbacks, S3 events  
  packages/  
    db/            Prisma schema + migrations + seed  
    contracts/     Zod schemas + generated OpenAPI + TS client  
    ui/            Design-system components (React, Tailwind, Radix)  
    config/        eslint, tsconfig, tailwind presets  
    ledger/        Money + double-entry library (pure TS, 100% test  
coverage)  
  infra/  
    terraform/     VPC, RDS, Redis, S3, CloudFront/Bunny, ECS/EKS, IAM, KMS  
    k8s/ or ecs/   Service definitions  
  docs/  
    adr/           Architecture decision records  
    runbooks/      On-call runbooks
```

Environment	Purpose	Data	Payments
local	Dev via docker compose (Postgres, Redis, MinIO, LiveKit, MailHog)	Seed data only	Processor sandbox
preview	Per-PR ephemeral deploy of web + api	Seed data	Sandbox
staging	Release candidate, mirrors prod config	Synthetic + anonymised	Sandbox
production	Live	Real	Live, restricted keys

Tooling: GitHub + Actions CI (lint, typecheck, test, build, migration check, OpenAPI diff); Renovate; Sentry; Datadog or Grafana Cloud; 1Password/Doppler for secrets (never in repo); trunk-based development, squash merges, conventional commits, semantic release tags. Production deploys require 1 approval and a green staging run.

Data model

PostgreSQL 16, one schema `app`, plus a separate database `pii` (KYC documents, legal names, tax IDs) with its own credentials and KMS key. Common columns (`id` `uuid` `pk`, `created_at`, `updated_at`, `deleted_at`) are omitted below.

```
-- IDENTITY
users(email citext unique, password_hash text null, email_verified_at, role
enum('fan','creator','agency','staff','admin'),
      status enum('active','suspended','banned','deleted'), handle citext
unique, display_name, avatar_media_id,
      country char(2), locale, totp_secret_enc bytea null, age_verified_at,
age_verification_method, last_login_at)
auth_identities(user_id fk, provider enum('google','apple'),
provider_user_id, unique(provider, provider_user_id))
sessions(user_id fk, refresh_token_hash, device, ip inet, user_agent,
expires_at, revoked_at)
user_blocks(blocker_id fk users, blocked_id fk users, unique(blocker_id,
blocked_id))

-- CREATORS & VERIFICATION
creator_profiles(user_id fk unique, bio, banner_media_id, category text[],
is_paid bool, subscription_price_cents int,
      currency, welcome_message_id null, dm_price_cents int default 0,
comments_enabled bool, geo_block char(2)[],
      status
enum('draft','pending_review','approved','rejected','suspended'),
approved_at, referral_code unique)
verifications(subject_user_id fk null, subject_performer_id fk null,
provider, provider_ref, type enum('id_liveness','age_estimate'),
      status enum('pending','approved','rejected','expired'), result_json
jsonb, reviewed_by fk users null, reviewed_at)
performers(creator_id fk, legal_name_ref (pii), stage_name, dob_ref (pii),
verification_id fk, release_doc_ref (pii), status)
content_performers(post_id fk null, media_id fk, performer_id fk) -- every
person in every media item
```

```
tax_profiles(user_id fk, form_type, country, tax_id_ref (pii), status,
submitted_at)
```

-- CONTENT

```
media_assets(owner_id fk users, kind enum('image','video','audio'),
storage_key, mime, bytes bigint, width, height, duration_ms,
    hls_key null, thumb_key null, blurred_key null, sha256, phash, status
enum('uploaded','processing','in_review','approved','rejected','failed'),
    moderation_case_id null)
vault_folders(creator_id fk, name, parent_id null)
vault_items(folder_id fk null, media_id fk, tags text[])
posts(creator_id fk, body text, visibility
enum('public','subscribers','ppv','tier'), price_cents int null, tier_id
null,
    status enum('draft','scheduled','in_review','published','removed'),
publish_at, pinned bool, like_count int, comment_count int)
post_media(post_id fk, media_id fk, position int, is_preview bool)
post_likes(post_id, user_id, pk(post_id,user_id))
comments(post_id fk, user_id fk, body, parent_id null, status)
bookmarks(user_id, post_id, pk(user_id, post_id))
bundles(creator_id fk, title, description, price_cents, currency,
quantity_limit int null, sold_count int, status)
bundle_items(bundle_id fk, post_id null, media_id null)
```

-- MONETISATION

```
subscription_plans(creator_id fk, tier_name, period_months int in (1,3,6,12),
price_cents, discount_pct, active bool)
promotions(creator_id fk, kind enum('trial','discount'), trial_days int,
discount_pct int, max_uses, used, starts_at, ends_at, code unique)
subscriptions(fan_id fk, creator_id fk, plan_id fk, promotion_id null, status
enum('trialing','active','past_due','cancelled','expired'),
    current_period_start, current_period_end, auto_renew bool,
cancelled_at, processor_sub_ref, unique(fan_id, creator_id) where status in
active set)
entitlements(user_id fk, resource_type
enum('post','message','bundle','stream','media'), resource_id uuid,
source_purchase_id fk,
    unique(user_id, resource_type, resource_id)) -- single source of
truth for "can view"
purchases(buyer_id fk, seller_id fk, type
enum('subscription','renewal','ppv_post','ppv_message','bundle','tip','stream
_ticket','stream_gift','paid_dm','wallet_topup'),
    resource_id null, gross_cents, currency, tax_cents, fee_cents,
```

```

net_cents, status
enum('pending','succeeded','failed','refunded','charged_back'),
    payment_id fk, idempotency_key unique)
payments(processor enum, processor_txn_ref unique, method, amount_cents,
currency, status, three_ds bool, risk_score, raw_jsonb)
wallets(user_id fk unique, balance_cents bigint, currency)    -- fan prepaid
balance (derived from ledger, cached)

-- LEDGER (append-only)
ledger_accounts(owner_type
enum('platform','creator','fan_wallet','processor','tax','reserve','referral'
,'agency'), owner_id null, currency, unique(owner_type, owner_id, currency))
ledger_transactions(kind, reference_type, reference_id, posted_at)
ledger_entries(transaction_id fk, account_id fk, amount_cents bigint)    --
signed; sum per transaction = 0 (DB constraint trigger)
creator_balances(creator_id, pending_cents, available_cents, updated_at)    --
materialised from ledger
payout_methods(creator_id fk, provider
enum('payoneer','paxum','bank_swift','bank_sepa','bank_ach','local'),
details_ref (pii), verified_at, is_default)
payouts(creator_id fk, method_id fk, amount_cents, currency, fx_rate, status
enum('requested','approved','processing','paid','failed','reversed'),
provider_ref, approved_by null)

-- MESSAGING
conversations(creator_id fk, fan_id fk, last_message_at, creator_unread int,
fan_unread int, unique(creator_id, fan_id))
messages(conversation_id fk, sender_id fk, sent_by_staff_id null, body,
price_cents null, is_mass bool, mass_message_id null, status, read_at)
message_media(message_id fk, media_id fk, position)
mass_messages(creator_id fk, audience_filter jsonb, body, price_cents,
scheduled_at, status, recipient_count, sent_count)

-- LIVE
live_streams(creator_id fk, title, access
enum('subscribers','ticketed','free_followers'), ticket_price_cents, status
enum('scheduled','live','ended','removed'),
    room_name, scheduled_at, started_at, ended_at, peak_viewers,
recording_media_id null, tip_goal_cents)
stream_chat(stream_id fk, user_id fk, body, tip_purchase_id null)

-- GROWTH & B2B
follows(follower_id, creator_id, pk)

```

```

referrals(referrer_creator_id fk, referred_creator_id fk unique, share_bps
int default 500, expires_at)
agencies(owner_user_id fk, legal_entity_ref (pii), status)
agency_creators(agency_id fk, creator_id fk, split_bps int, status
enum('invited','active','ended'), creator_consented_at)
agency_members(agency_id fk, user_id fk, role
enum('owner','manager','chatter'), scopes text[])

-- TRUST & SAFETY / OPS
reports(reporter_id null, target_type, target_id, reason enum, details,
status, assigned_to null)
moderation_cases(target_type, target_id, source
enum('upload','report','classifier','hash_match','dmca','ncii'),
signals jsonb, priority int, status
enum('open','escalated','actioned','dismissed'), decision, decided_by,
decided_at)
legal_reports(case_id fk, authority enum('NCMEC','IWF','police','other'),
external_ref, filed_at)
dmca_notices(claimant, target_ids uuid[], status, counter_notice_at)
audit_log(actor_id, actor_type, action, target_type, target_id, ip, metadata
jsonb) -- append-only, retained 7 years
notifications(user_id fk, type, payload jsonb, read_at)
notification_prefs(user_id fk, type, channel enum('in_app','email','push'),
enabled)

```

Indexes (minimum): posts(creator_id, status, publish_at desc),
subscriptions(fan_id, status), subscriptions(creator_id, status),
entitlements(user_id, resource_type, resource_id) unique,
messages(conversation_id, created_at desc), purchases(seller_id, created_at),
ledger_entries(account_id, transaction_id), media_assets(sha256),
media_assets(phash) (for near-duplicate lookup), trigram index on users.handle and
users.display_name.

API specification

REST under `https://api.lumora.app/v1`; the OpenAPI 3.1 file generated from `packages/contracts` is canonical, and this list is the minimum it must contain.

Auth: short-lived JWT access token (15 min, in memory) + rotating refresh token (30 days, httpOnly Secure SameSite=Lax cookie). Staff/admin endpoints require SSO + hardware-key MFA and live on a separate host. Agency chatters receive tokens scoped by

`agency_members.scopes`.

Errors: RFC 9457 problem+json {type, title, status, code, detail, requestId}. Stable code values, e.g. AGE_VERIFICATION_REQUIRED, KYC_REQUIRED, INSUFFICIENT_FUNDS, PAYMENT_DECLINED, NOT_ENTITLED, GEO_BLOCKED, RATE_LIMITED.

Pagination: ?cursor=&limit= (max 50) → {data:[], nextCursor}. **Idempotency:** Idempotency-Key header required on all purchase, tip, payout and refund POSTs; replays return the stored response for 24 h.

Rate limits: 300 req/min per user, 20 login attempts/hour per IP, 10 purchases/min per user, mass messages 5/hour per creator.

Module	Method & path	Notes
Auth	POST /auth/signup POST /auth/login POST /auth/refresh POST /auth/logout	Email + password; returns access token
Auth	POST /auth/oauth/google POST /auth/magic-link POST /auth/verify-email	
Auth	POST /auth/2fa/setup POST /auth/2fa/verify DELETE /auth/2fa	TOTP; mandatory for creators
Account	GET/PATCH /me GET /me/sessions DELETE /me/sessions/:id POST /me/export DELETE /me	Data export + deletion
Age	POST /age-verification/session GET /age-verification/status	Returns vendor SDK token
Creator onboarding	POST /creator/apply POST /creator/kyc/session GET /creator/status PUT /creator/tax-profile	
Performers	POST /creator/performers POST /creator/performers/:id/kyc-invite GET /creator/performers	Co-performer verification by link

Module	Method & path	Notes
Profiles	GET /creators/:handle PATCH /creator/profile POST /creators/:id/follow DELETE /creators/:id/follow	Public payload hides locked media
Discovery	GET /search?q=&category= GET /creators/featured GET /categories	
Media	POST /media/uploads → presigned multipart URLs; POST /media/:id/complete; GET /media/:id/playback	Playback returns signed HLS URL only if entitled
Vault	GET/POST /vault/folders GET /vault/items?folder=&tag= PATCH /vault/items/:id	
Posts	POST /posts PATCH /posts/:id DELETE /posts/:id GET /creators/:id/posts GET /feed	Visibility + price + publishAt
Engagement	POST/DELETE /posts/:id/like GET/POST /posts/:id/comments POST/DELETE /posts/:id/bookmark	
Plans	GET/PUT /creator/plans POST /creator/promotions GET /creator/promotions	
Subscriptions	POST /subscriptions {creatorId, planId, promoCode?, paymentMethod} PATCH /subscriptions/:id (auto- renew) GET /me/subscriptions	
Purchases	POST /purchases {type, resourceId, paymentSource: wallet	card}
Tips	POST /tips {creatorId, amountCents, context: post	message
Wallet	GET /wallet POST /wallet/topups GET /wallet/transactions PUT /wallet/limits	Fan spend limits

Module	Method & path	Notes
Bundles	POST/PATCH /creator/bundles GET /creators/:id/bundles	
Messaging	GET /conversations GET /conversations/:id/messages POST /conversations/:id/messages POST /messages/:id/read	Locked message body redacted until unlocked
Mass messaging	POST /creator/mass-messages GET /creator/mass-messages/:id GET /creator/audiences/preview	
Live	POST /creator/streams POST /creator/streams/:id/start POST /creator/streams/:id/end POST /streams/:id/join	join returns LiveKit token if entitled
Analytics	GET /creator/analytics/summary?from=&to= GET /creator/analytics/earnings GET /creator/analytics/fans GET /creator/statements.csv	
Payouts	GET/POST /creator/payout-methods GET /creator/balance POST /creator/payouts GET /creator/payouts	
Referrals	GET /creator/referrals	
Agency	POST /agencies POST /agencies/:id/invites POST /creator/agency-invites/:id/accept GET /agencies/:id/creators POST/DELETE /agencies/:id/members GET /agencies/:id/earnings	
Reports	POST /reports POST /takedown-requests (public, no login) POST /dmca	

Module	Method & path	Notes
Notifications	GET /notifications POST /notifications/read GET/PUT /notification-prefs POST /push/subscriptions	
Webhooks (inbound)	POST /webhooks/processor/:name POST /webhooks/kyc/:name POST /webhooks/media POST /webhooks/livekit POST /webhooks/payouts/:name	Signature verified, stored raw, processed async
Admin (separate host)	/admin/kyc-queue /admin/moderation-cases /admin/users/:id /admin/refunds /admin/payouts /admin/legal-reports /admin/audit-log	Every call audited

WebSocket `wss://api.lumora.app/v1/ws` (JWT on connect). Server events: `message.created`, `message.read`, `conversation.updated`, `notification.created`, `purchase.succeeded`, `stream.status`, `stream.tip`, `typing`. Client events: `typing.start`, `typing.stop`, `presence.ping`. Backed by Redis pub/sub so any API node can deliver to any socket.

Critical flows

These five flows carry money or legal risk; each must have an end-to-end test in staging before launch.

A. Creator signup → approved

1. User signs up, verifies email, sets up TOTP 2FA (blocking).
2. `POST /creator/apply` creates `creator_profiles(status=draft)`.
3. `POST /creator/kyc/session` → vendor SDK runs ID + liveness in browser.
4. Vendor webhook → `verifications` updated. Reject if age < 18, doc mismatch, or sanctions hit. Auto-approve only when vendor confidence ≥ threshold; else route to admin KYC queue.
5. Creator submits tax profile and payout method.

6. Status → `pending_review` ; admin approves within SLA (24 h). Status → `approved` ; profile goes live; first 10 posts force human review.

B. Fan subscribes

1. Fan opens creator page; if not age-verified and region requires it → AV flow first (`AGE_VERIFICATION_REQUIRED`).
2. `POST /subscriptions` with `Idempotency-Key` . API checks geo-block, block list, duplicate active sub.
3. Create `purchases(status=pending)` ; call processor (3-D Secure if challenged).
4. On success webhook: in one DB transaction → `payments` succeeded, `purchases` succeeded, `subscriptions` active with period end, post ledger transaction (fan card → creator pending 80%, platform fee 20%, tax), grant subscription entitlement.
5. Emit `purchase.succeeded` ; send welcome message if set; notify creator.
6. Renewal job runs hourly: charges subs whose `current_period_end` < now+1h; on failure → `past_due` , retry at +1d, +3d, +5d, then `expired` and entitlements revoked.

C. PPV unlock (post or message)

1. Fan sees blurred preview + price. `POST /purchases {type: ppv_post|ppv_message, resourceId, paymentSource}` .
2. Wallet path: lock wallet row (`SELECT ... FOR UPDATE`), check balance and fan spend limit, debit via ledger, create entitlement — all in one transaction.
3. Card path: as flow B steps 3–4.
4. Client re-fetches resource; media endpoints now return signed URLs.

D. Upload → moderation → publish

1. `POST /media/uploads` → presigned multipart S3 URLs (private bucket, 15-min expiry).
2. Client uploads directly; `POST /media/:id/complete` .
3. Worker: compute SHA-256 + perceptual hash → check against known-CSAM hash services and internal ban list. Match → quarantine, freeze account, open P0 case, file legal report; creator is not told the reason.
4. Run classifiers (nudity, apparent minor, violence, prohibited categories, face detection). For each detected face require a linked verified performer; unmatched face → human review.
5. Transcode video to HLS (360p/720p/1080p), generate thumbnail + blurred preview.
6. Decide: auto-approve (low risk + established creator) or human queue. Only `approved` media can attach to a published post.

7. Scheduled posts publish via a delayed job at `publish_at` ; subscribers notified.

E. Creator payout

1. Earnings sit in `pending` for the hold period (7 days new creators, 3 days after 90 days in good standing), then a nightly job moves them to `available` .
2. Creator requests payout (min \$20) or auto-payout weekly.
3. Checks: KYC valid, tax profile present, no open P0 case, no chargeback spike, payout method verified ≥ 72 h ago (or re-verified).
4. Payouts > \$10k or flagged $\rightarrow$ manual approval. Ledger: creator available $\rightarrow$ payout in-transit.
5. Payout provider API call; webhook marks `paid` or `failed` (failed reverses ledger entry and notifies creator).
6. Monthly statement PDF generated per creator.

Media, live streaming, messaging

Media pipeline

- Buckets: `uploads-raw` (private, 7-day lifecycle), `media-processed` (private), `media-public` (avatars, banners, blurred previews only).
- Limits: image 50 MB, video 5 GB / 4 h, audio 500 MB. Allowed types checked by magic bytes, not extension. EXIF/GPS stripped on all images.
- Video: HLS with AES-128 segment keys; key endpoint checks entitlement on each request. Signed CDN URLs, 10-min TTL, bound to user ID.
- Forensic watermark: visible handle overlay rendered client-side on images/video, plus server-side invisible watermark on downloads where the vendor supports it.
- Image variants: 320, 640, 1280, 2048 px WebP/AVIF, generated on upload.
- Cost guardrails: per-creator storage quota (default 500 GB), per-fan bandwidth anomaly alerts (scraping detection).

Live streaming

- LiveKit (self-hosted on dedicated nodes, or LiveKit Cloud if its acceptable-use policy permits) with WHIP/RTMP ingress for OBS and browser publishing.
- `POST /streams/:id/join` mints a viewer token only for entitled users; token TTL = 2 h; kick on subscription expiry.

- Real-time moderation: sample a frame every 10 s to the classifier; P0 signal → auto-end stream + case.
- Tips during stream post a `stream.tip` event for on-screen alerts; tip-goal progress computed server-side.
- Recording via LiveKit Egress to `media-processed` ; goes through flow D before appearing in vault.
- Capacity target v1: 200 concurrent streams, 2,000 viewers per stream.

Messaging

- Messages stored in Postgres; delivered via WebSocket; offline → push + email digest.
- Locked (PPV) message: body and media hidden until entitlement exists; preview text and blurred thumbnail allowed.
- Mass message fan-out: worker paginates audience in batches of 1,000, inserts messages, publishes events; rate limited; creator sees progress.
- Chatter actions record `sent_by_staff_id` ; chatters cannot see payout data or export fans.
- Anti-abuse: link scanning, profanity/threat classifier, fans can mute or block, creators can restrict who may message (subscribers only, min spend).
- Retention: messages kept while both accounts exist; deleted 30 days after account deletion unless under legal hold.

Payments, ledger and payouts engine

All money movement goes through `packages/ledger` ; no service writes balances directly. The ledger is append-only and every transaction sums to zero.

Processor abstraction (`PaymentProvider` interface): `createCharge` , `createRecurring` , `cancelRecurring` , `refund` , `getStatus` , `verifyWebhook` , `tokenizeCard` (hosted fields/iframe only — card data never touches our servers, keeping PCI scope at SAQ-A). Two adapters at launch (primary + failover); a router picks by BIN country, amount and recent approval rate.

Ledger postings (examples, \$10.00 PPV, 20% fee, VAT ignored for brevity)

Account	Entry (cents)
<code>processor_clearing</code>	+1000

Account	Entry (cents)
creator:{id}:pending	-800
platform:revenue	-200

On settlement the processor clearing account is reconciled against the processor's daily settlement file; differences open a finance alert. Referral share: on each posting from a referred creator, move 5% of the fee from `platform:revenue` to `referral:{referrerId}:pending`. Agency split: from creator net, move `split_bps` to `agency:{id}:pending` on the same transaction.

Refunds and chargebacks: full or partial refund reverses the original postings proportionally and revokes the entitlement. Chargeback: reverse postings, debit creator (balance may go negative and is recovered from future earnings), add a chargeback fee line, flag the fan account; 2+ chargebacks → fan blocked from card payments.

Taxes: tax engine called at checkout with fan country (IP + card BIN + billing country must agree on 2 of 3 per EU VAT rules); tax stored on purchase and posted to `tax:{jurisdiction}`. Creator tax: 1099-NEC/1099-K data export for US creators; DAC7 report for EU creators annually.

FX: fans are charged in their display currency where the processor supports it; creator balances are held in USD. FX rate captured on the purchase row.

Reconciliation jobs (daily): processor settlement vs ledger; payout provider vs ledger; sum of all creator balances = liability account; any mismatch pages finance on-call.

Fraud rules (v1): 3DS on first card use and on charges > \$50; velocity limits (5 cards per account, 3 failures/hour → lock); device fingerprint and IP reputation; new account + high spend → manual review; creator self-dealing detection (fan and creator sharing device/IP/card).

Trust & Safety, security, privacy

No media is visible to anyone but its uploader until it passes moderation; this rule is enforced in the playback endpoint, not just the UI.

Moderation system

- Signals feed `moderation_cases` with a priority: P0 (hash match / apparent minor / non-consent report) → 15-min SLA, 24/7; P1 (violence, prohibited category) → 4 h; P2 (spam, IP, profile issues) → 48 h.
- Admin console queues: KYC review, media review (side-by-side with performer IDs), reports, DMCA, NCII takedowns, payouts, refunds. Reviewer tools: blur-by-default, grayscale toggle, time-limited viewing, wellness breaks (moderator welfare).
- Actions: approve, remove, restrict, suspend, ban, freeze payouts, file legal report. Each action needs a reason code and is written to `audit_log`.
- Public takedown form (no login) for anyone depicted without consent; P0 priority; content hidden immediately pending review.
- Repeat-offender graph: link accounts by device, IP, payment instrument, face embedding (performers only, with consent) to stop ban evasion.

Age assurance

- Region rules table (`country/state` → `required method`) maintained by legal; middleware enforces on every content and purchase endpoint.
- Methods supported: ID document, facial age estimation, credit-card check, reusable digital ID where available.

Application security

- OWASP ASVS L2 checklist in PR template; Semgrep + dependency scanning in CI; container image scanning.
- Argon2id for passwords; breached-password check (k-anonymity API); TOTP + WebAuthn for creators; login anomaly alerts.
- Authorization via a policy layer (CASL or Oso) — every resource read checks owner, entitlement, block list, geo-block, moderation status.
- CSP, HSTS, SameSite cookies, CSRF tokens on cookie-auth routes; strict CORS.
- Secrets in AWS Secrets Manager; KMS envelope encryption for `pii` DB columns; no PII in logs (log scrubber).
- Admin access: SSO + hardware keys, IP allow-list, just-in-time elevation, session recording for PII views.
- Pen test before public launch and yearly; bug bounty after launch.

Privacy

- Data map + DPIA maintained in `/docs/privacy`. Lawful basis per data type.

- Self-serve export (JSON + media zip) and deletion; deletion job removes data in 30 days except records required by law (KYC, 2257-style records, tax, ledger: retained per statute in pii with restricted access).
- Cookie consent with reject-all; analytics (PostHog self-hosted) without third-party ad trackers.
- Fan identities never exposed to creators beyond handle/display name; creators' legal names never exposed to fans.

Frontend: routes, screens, components

Next.js App Router, React Server Components for public pages, client components for interactive surfaces; TanStack Query against the generated API client; Tailwind + Radix UI in packages/ui . Mobile-first, installable PWA, WCAG 2.2 AA.

Route	Screen	Access
/	Marketing landing (original design)	Public, SFW
/signin /signup /reset	Auth	Public
/age-check	Age gate + AV handoff	Public
/explore	Search, categories, featured creators	Logged in, AV
/[handle]	Creator profile: header, plans, posts grid, bundles, follow/subscribe	Public SFW preview; full when entitled
/p/[postId]	Post detail, comments, unlock	Entitled / unlock
/feed	Fan home feed	Fan
/messages /messages/[id]	Inbox, chat thread, unlock in chat, tip	Logged in
/live/[streamId]	Stream player, chat, tips, goal bar	Entitled
/wallet	Balance, top-up, history, limits	Fan
/subscriptions	Manage subs, auto-renew, cancel	Fan

Route	Screen	Access
/settings/*	Profile, security (2FA, sessions), notifications, privacy, data export, delete	Logged in
/creator/onboarding	Stepper: apply → KYC → tax → payout → review	Creator
/creator/dashboard	Earnings cards, activity, upcoming lives	Creator
/creator/posts/new	Composer: media from upload or vault, visibility, price, schedule, tag performers	Creator
/creator/vault	Folders, tags, bulk upload, per-item stats	Creator
/creator/messages/mass	Audience builder, preview, schedule	Creator
/creator/pricing	Plans, bundles, promotions/trials	Creator
/creator/live	Go-live setup, OBS keys, browser publish	Creator
/creator/analytics	Charts, top fans, statements export	Creator
/creator/payouts	Balance, methods, history	Creator (no chatter access)
/creator/performers	Co-performers, invite to verify	Creator
/creator/referrals	Link, referred creators, earnings	Creator
/agency/*	Creators list, members/chatters, earnings, audit	Agency
admin.lumora.internal/*	All T&S and ops queues	Staff, SSO

Shared components: LockedMedia (blur + price + unlock CTA), MediaPlayer (HLS player with watermark overlay), Composer, PriceInput (minor units, currency), CheckoutSheet (wallet vs card, 3DS iframe), TipSheet, ChatThread (virtualised), AudienceBuilder, StatCard, DataTable, Stepper, ReportDialog, AgeGate.

Design deliverables before build: brand identity (logo, palette, type), Figma design system mapped 1:1 to `packages/ui`, high-fidelity screens for every route above in mobile + desktop, empty/loading/error states, email templates.

Non-functional requirements, testing, observability

Area	Requirement
Availability	99.9% monthly for API + web; payments and playback are tier-1
Latency	p95 API < 300 ms reads, < 800 ms purchases (excluding 3DS); first video frame < 2.5 s on 4G
Scale (v1 design point)	500k registered users, 50k DAU, 5k concurrent sockets per node, 2M messages/day, 200 live streams
Durability	RDS multi-AZ, PITR 35 days, daily snapshots copied cross-region; media in S3 with versioning on processed bucket
Recovery	RPO 5 min, RTO 1 h; DR drill quarterly
Security	See previous section; zero critical findings at launch pen test
Accessibility	WCAG 2.2 AA; keyboard + screen reader on all purchase flows
Localization	i18n-ready (ICU messages); launch in English, add ES, PT-BR, DE, FR in phase 3
Browser support	Last 2 versions of Chrome, Safari (iOS + macOS), Firefox, Edge

Testing strategy

- Unit: Vitest; `packages/ledger` and pricing/tax maths at 100% branch coverage.
- Integration: API against real Postgres/Redis via Testcontainers; processor and KYC vendors mocked with recorded contract fixtures.
- End-to-end: Playwright for flows A-E (see Critical flows) on every merge to main against preview env; processor sandbox in staging nightly.

- Load: k6 scripts for feed, playback token, purchase, mass message fan-out; run before each release.
- Security: DAST (OWASP ZAP) on staging weekly; authorization test matrix (every role × every endpoint).
- Moderation pipeline: test with vendor-provided safe test hashes only — never real illegal material.

Observability

- OpenTelemetry traces across web → api → worker; structured JSON logs with request ID; PII scrubbed.
- Business dashboards: GMV, approval rate per processor, chargeback rate, refund rate, KYC queue age, moderation SLA, payout success.
- Alerts (PagerDuty): error rate > 1% for 5 min, payment approval drop > 10 points, P0 moderation case older than 15 min, ledger reconciliation mismatch, webhook backlog > 1,000.

Backlog: epics, stories, acceptance criteria, sprints

18 epics; import each story as a ticket (Jira/Linear) with its acceptance criteria. Estimates are in ideal engineer-weeks.

#	Epic	Est.	Phase
E0	Platform foundations (repo, CI/CD, infra, auth skeleton, design system)	10	MVP
E1	Accounts, auth, 2FA, sessions	4	MVP
E2	Age assurance + region rules	3	MVP
E3	Creator onboarding, KYC, performers, tax	6	MVP
E4	Media upload + processing pipeline	7	MVP
E5	Moderation pipeline + admin console	10	MVP
E6	Profiles, posts, feed, engagement	6	MVP
E7	Payments abstraction, ledger, wallet	10	MVP
E8	Subscriptions, plans, promotions	5	MVP

#	Epic	Est.	Phase
E9	PPV, bundles, tips	4	MVP
E10	Messaging + mass messages	6	MVP
E11	Vault	2	MVP
E12	Payouts + statements	5	MVP
E13	Notifications	3	MVP
E14	Live streaming	7	Parity
E15	Analytics dashboards	4	Parity
E16	Discovery + search	3	Parity
E17	Referrals + agencies + chatter seats	6	Parity

Representative stories with acceptance criteria (write the rest in the same format)

- ☐ **E3-1 Creator completes KYC.** Given an applicant with verified email and 2FA, when they finish the vendor flow, then `verifications.status` updates within 60 s of the webhook; age < 18 or document mismatch rejects; vendor score below threshold lands in the admin queue; applicant sees status on `/creator/onboarding`.
- ☐ **E4-2 Upload large video.** A 4 GB MP4 uploads via multipart with resume after network loss; HLS renditions at 360/720/1080p exist within 15 min; original raw file deleted within 7 days; asset stays invisible to others until `approved`.
- ☐ **E5-1 Hash match quarantine.** A test-hash match quarantines the asset, suspends the uploader's payouts, opens a P0 case and pages on-call within 1 min; creator UI shows a generic "under review" message only.
- ☐ **E5-4 Face ↔ performer check.** A media item with 2 detected faces and only 1 linked verified performer cannot auto-approve and goes to human review.
- ☐ **E7-1 Ledger invariant.** Any attempt to commit a ledger transaction whose entries do not sum to 0 fails at the DB layer; property-based tests cover 10k random postings.
- ☐ **E7-3 Idempotent purchase.** Two identical `POST /purchases` with the same key produce exactly one charge and one entitlement; the second returns the first response.
- ☐ **E8-2 Renewal retry.** A failed renewal sets `past_due`, retries at +1, +3, +5 days, then expires and revokes access; fan receives email at each step.

- ☐ **E9-1 Unlock PPV from wallet.** With sufficient balance, unlock completes < 800 ms p95; balance, creator pending balance and platform revenue change by exactly the expected cents; insufficient funds returns `INSUFFICIENT_FUNDS`.
- ☐ **E10-3 Mass message to 50k fans.** Delivery completes < 10 min; blocked fans and fans who muted the creator are excluded; locked message shows blurred preview + price.
- ☐ **E12-2 Payout request.** Blocked if KYC expired, tax profile missing, open P0 case, or payout method changed < 72 h ago; otherwise ledger moves available → in-transit and provider call is made; failure reverses.
- ☐ **E14-1 Ticketed live stream.** Only ticket holders receive a join token; a fan whose subscription lapses mid-stream is disconnected within 60 s; tips appear on stream within 2 s.
- ☐ **E17-2 Chatter permissions.** A chatter can read/send messages for assigned creators only; payouts, statements and fan export return 403; every message stores `sent_by_staff_id`.

Sprint plan (2-week sprints, team of ~10 engineers)

Sprint	Weeks	Focus	Demo at end
S1-S3	1-6	E0, E1, design system	Sign up, log in with 2FA on staging
S4-S5	7-10	E2, E3, E4	Creator verifies identity and uploads video
S6-S7	11-14	E5, E6, E11	Moderated post published to profile
S8-S9	15-18	E7, E8, E9	Fan subscribes and unlocks PPV in processor sandbox
S10-S11	19-22	E10, E12, E13	Messages, mass message, first sandbox payout
S12	23-24	Hardening, pen test fixes, load tests	Closed beta with 100 creators
S13-S15	25-30	E14, E15	Live stream with tips; analytics
S16-S17	31-34	E16, E17	Discovery, referrals, agencies

Sprint	Weeks	Focus	Demo at end
S18–S20	35–40	Second processor, VAT engine, scale + polish	Public launch

Before sprint 1, the business must provide: chosen entity and launch markets, signed processor and KYC contracts (or sandbox access), counsel-approved Acceptable Use Policy and content rules, brand and domain.